

Lab 7: Neural Networks

DT8008 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Anton Bauer, antibau25@student.hh.se, TAEIS 25

March 2026

Introduction

In this lab we explored non-linear classification using Neural Networks to recognize handwritten digits. In Part A, we loaded the dataset, visualized it with PCA, and implemented feedforward propagation along with unregularized and regularized cost functions to evaluate pre-trained weights. In Part B, we implemented the backpropagation algorithm to learn the neural network parameters from scratch using gradient descent, evaluated the model's accuracy under regularization, and visualized the learned representations of the hidden layer units.

Part A: Feedforward Propagation and Cost Function

Objective:

Part A objective was to understand the structure of a neural network by implementing the feedforward propagation algorithm. We used given pre-trained weights to predicted handwritten digits (from 0 to 9) from a dataset of 5000 images and computed the cost functions to measure the model's performance.

Key Concept:

1. **Neural Networks:** Neural networks can directly form complex non-linear hypotheses as they pass data through multiple layers (input layer, hidden layer/layers, and output layer) allowing them to learn features themselves; which sets them apart from simple linear classifiers, such as logistic regression, that needs techniques like feature mapping to learn non-linear relations.
2. **Feedforward Propagation:** Is the process of calculating the output of the neural network. The inputs $a^{(1)}$ are multiplied by a parameter matrix $\Theta^{(1)}$, passed through a sigmoid activation function to form the hidden layer activations $a^{(2)}$, and then passed through $\Theta^{(2)}$ to compute the final output probabilities $a^{(3)}$.
3. **One-hot Vector:** A method for encoding categorical target labels into a binary vector format. In this lab instead of representing a digit class as a single scalar value (like 3), it is mapped to a 10-dimensional vector where the index corresponding to the target class is set to 1, and all other elements are 0 (for example $[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$). This aligns the true labels with the network's 10 output units and allows the model to predict the probability for each individual class.
4. **Cost Function:** A function, $E(\Theta)$, that computes the error between the network's predicted outputs and the true encoded class labels (one-hot vectors). Because our output layer uses a

sigmoid activation function, this lab utilizes the Sum of Binary Cross-Entropies. This effectively treats the network's output as 10 independent logistic regression classifiers, computing and summing the binary log-loss for each individual digit class. As seen here:

$$E(\Theta) = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K \left[-y_k^{(i)} \log((h_{\Theta}(x^{(i)}))_k) - (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right],$$

where n is the total number of training examples (5,000 images); K is the total number of target classes/labels (10 digits); and $(h_{\Theta}(x^{(i)}))_k$ is the network's predicted probability that the i -th data point belongs to the k -th class.

5. **Regularization (λ):** A penalty term added to the cost function. It penalizes large parameter weights to prevent the model from overfitting the training data; the bias units (the first column of the Θ matrices) are excluded from regularization.

Methods/Algorithm:

Algorithm 1 Feedforward Propagation and Cost Computation

- 1: **Step 1: Dataset Loading and Visualization**
 - 2: Load the 5000×400 digits dataset (each row is an "unrolled" 20×20 pixel image) and visualize the data using PCA.
 - 3: **Step 2: Input Layer Preparation**
 - 4: Prepare the input layer $a^{(1)}$ by transposing the dataset X and appending a row of ones to account for the bias unit.
 - 5: **Step 3: Feedforward Implementation**
 - 6: Compute the hidden layer: $z^{(2)} = \Theta^{(1)}a^{(1)}$ and $a^{(2)} = \text{sigmoid}(z^{(2)})$. Add a bias row of ones to $a^{(2)}$.
 - 7: Compute the output layer: $z^{(3)} = \Theta^{(2)}a^{(2)}$ and $a^{(3)} = \text{sigmoid}(z^{(3)})$.
 - 8: **Step 4: One-Hot Encoding**
 - 9: Convert the true labels y into a matrix of one-hot encoded vectors Y .
 - 10: **Step 5: Unregularized Cost**
 - 11: Calculate the unregularized cost by summing the logarithmic errors between the predictions $a^{(3)}$ and true labels Y .
 - 12: **Step 6: Regularized Cost**
 - 13: Add the regularization term $\frac{\lambda}{2n} \sum (\Theta)^2$ (excluding the bias columns) to the unregularized cost to penalize complex weights.
-

Observation and Results:

Before training, the lab had us examine the 400-dimensional image data using Principal Component Analysis (PCA) as shown in Figure 1. This provided an underlying view of how different classes of digits group together.

Using the provided pre-trained parameters ($\Theta^{(1)}$ and $\Theta^{(2)}$), we ran the feedforward algorithm on the 5000 images. The implementation gave an unregularized cost of approximately 0.15835, matching the expected cost given in the lab description.

Furthermore, by taking the highest probability from the output layer ($a^{(3)}$) for each image, we predicted the final class labels. The result was a classification accuracy of 98.62%. Achieving this high accuracy confirms that our feedforward code worked correctly and that we could use the given pre-trained weights to classify the digits.

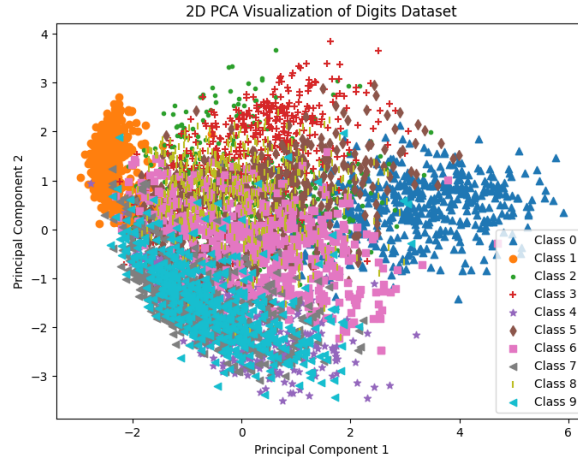


Figure 1: Visualizing the digits dataset with PCA. The scatter plot shows the high-dimensional layout of the different handwritten digits (0-9) compressed into 2 principal components.

Part B: Backpropagation and Model Training

Objective:

The objective of Part B was to implement the backpropagation algorithm to calculate gradients and learn the neural network weights from scratch. After training the network with iterative gradient descent, we evaluated the accuracy using different regularization bounds and extracted the hidden units to visually interpret the features the network had learned.

Key Concepts:

1. **Backpropagation:** An algorithm that computes the gradient of the cost function by propagating the error backwards from the output layer to the hidden layer. It calculates the error terms $\delta^{(3)}$ and $\delta^{(2)}$ to derive the gradients $\Delta^{(1)}$ and $\Delta^{(2)}$ used to adjust the Θ matrices.
2. **Hidden Layer Representation:** A neural network is a "black box" but the weights mapping the inputs to the hidden layer can be found. By reshaping the 400 weights of a single hidden unit back into a 20×20 grid (original image format), we can visualize the specific shapes (like strokes or edges) that the node activates upon.

Methods/Algorithm:

Algorithm 2 Neural Network Training with Backpropagation

- 1: **Step 1: Random Initialization**
 - 2: Initialize $\Theta^{(1)}$ and $\Theta^{(2)}$ with random small weights to break symmetry and allow the network to learn different features.
 - 3: **Step 2: Iterative Optimization**
 - 4: Loop over a set number of iterations (like *max_iterations* = 2000).
 - 5: **Step 3: Forward Pass and Error Computation**
 - 6: Perform feedforward to get predictions $a^{(3)}$. Compute the output error $\delta^{(3)} = 2(a^{(3)} - Y)$.
Note: While the theoretical derivative of the Cross-Entropy loss yields $\delta^{(3)} = a^{(3)} - Y$ (as defined in the lab manual), the provided Python implementation includes a multiplier of 2 (`delta3 = 2 * (a3 - Y)`). Unsure if intentional, but it doubles the gradient, allowing the network to converge faster within the 2000-iteration limit.
 - 7: **Step 4: Backward Pass (Backpropagation)**
 - 8: Compute the hidden layer error $\delta^{(2)}$ by multiplying $\delta^{(3)}$ with $\Theta^{(2)}$ and the derivative of the sigmoid function.
 - 9: Compute the unregularized gradients $\Delta^{(1)}$ and $\Delta^{(2)}$.
 - 10: **Step 5: Apply Regularization**
 - 11: Add the regularization term $\frac{\lambda}{n} \Theta_{ij}^{(l)}$ to the gradients for $j \geq 1$ to prevent overfitting.
 - 12: **Step 6: Update Parameters**
 - 13: Update the weights using gradient descent: $\Theta = \Theta - \alpha \Delta$.
 - 14: **Step 7: Evaluation and Visualization**
 - 15: Use the trained weights to predict the training set labels and calculate the final accuracy. Finally, reshape and plot the hidden units as 2D images.
-

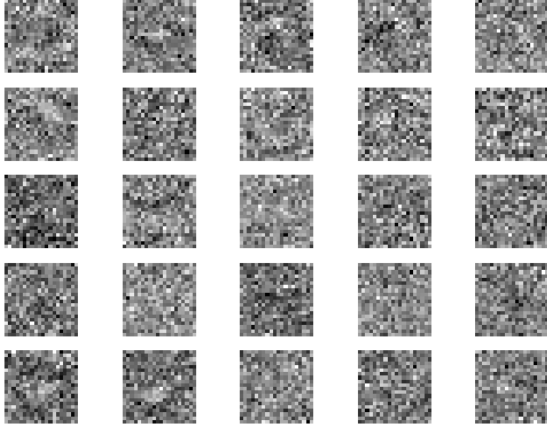
Observation and Results:

First we trained the neural network with a 25 hidden units for max 2000 iterations and with a learning rate of $\alpha = 0.5$. With regularization applied ($\lambda = 20$), the network achieved a training accuracy of approximately 93.86%. The same but unregularized network ($\lambda = 0$) achieved a higher training accuracy (always over 95%), but the regularized model is still considered better. The higher training accuracy of the unregularized model is typically a sign of overfitting, meaning the network probably relied too heavily on specific parameters to memorize the training set. By applying a higher λ , the network is penalized for complex weights and forced to generalize better, ensuring it performs well on unseen test data also.

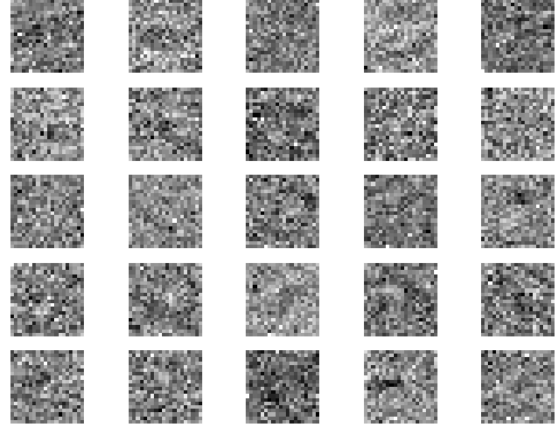
To see the representations captured by our 25 hidden units, we used the provided function that removed the bias terms and reshaped their 400-dimensional parameter vectors back into 20×20 pixel images. To see how our hyperparameters affect learning, we looked at four different scenarios, as shown in Figure 2.

As seen in Figure 2, the hidden units roughly correspond to distinct visual detectors. Looking at the four quadrants, we can observe the following impacts:

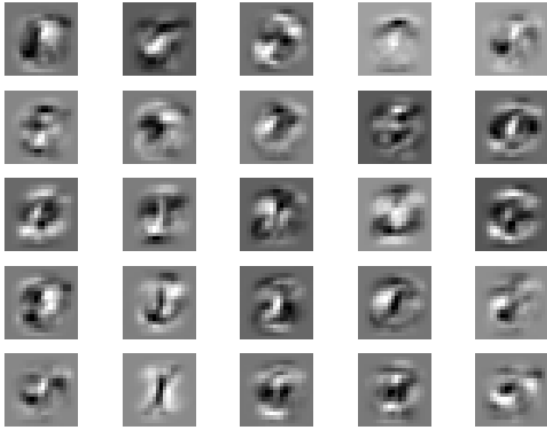
- **Baseline/Overfitting (Top Left):** With no penalty ($\lambda = 0$), the network learns highly complex noisy features. It essentially memorizes the pixel-level noise of the training set rather than learning general shapes.
- **Low Regularization (Top Right):** A small penalty ($\lambda = 1$) is still too weak to prevent overfitting in this scenario. The resulting hidden units still largely resemble static noise.
- **High Regularization (Bottom Left):** Applying a heavy penalty ($\lambda = 100$) forces the network to drop the complex pixel noise. This reveals the true intelligible output, where the units form clear distinct strokes and edges.



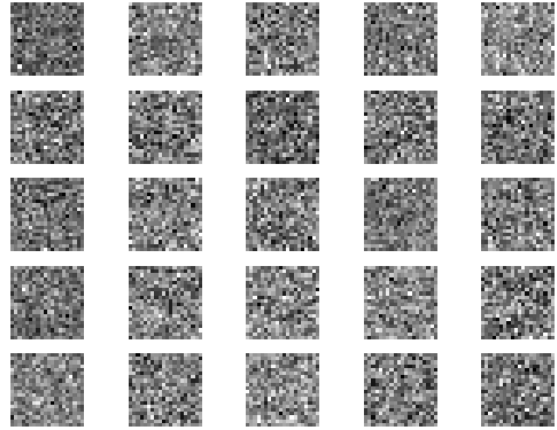
(a) Baseline / Overfitting ($\lambda = 0$, Max Iters=2000)



(b) Low Regularization ($\lambda = 1$, Max Iters=2000)



(c) High Regularization ($\lambda = 100$, Max Iters=2000)



(d) Underfitting ($\lambda = 1$, Max Iters=200)

Figure 2: Visualizing the 25 hidden units of the trained neural network under different iterations and regularization parameters (λ). The visualizations reveal how stroke/edge detectors change with hyperparameter adjustments.

- **Underfitting (Bottom Right):** Stopping the training prematurely (only 200 iterations) prevents the network from properly converging, resulting in underdeveloped features.

Conclusion

This lab showed the usefulness of Neural Networks in solving complex and non-linear classification problems like image recognition. In Part A, we learned the architecture of the feedforward process and cost functions by using provided pre-trained weights. In Part B, we learned to implementing backpropagation from scratch to calculate the gradients and learn the previously provided weights by ourselves. We also played around with tuning hyperparameters like λ (to prevent overfitting) and *max_iterations* (how many times is gradient descent allowed to run max), and finishing with visualizing the hidden layer to get an intuitive look at how the network decomposes images into patterns.